

A SMALL THEME FOR MODERN SLIDES

Odoo Core Development

Install, modules, fields, ORM, and environments

Weblearns Academy
weblearns@training.local
Odoo Full-Stack Developer Program
Lecture materials / July 2026



Agenda

- 1 Install, CLI, and configuration
- 2 Modules, manifests, and addons paths
- 3 Field design and relational models
- 4 ORM overrides and recordset APIs

TOC

1 A theme for modern talks

How this deck is taught

AWESOME-STYLE FRAME

Number rails, angled title art, and section slides are built in.

METROPOLIS DEFAULTS

Progress bars, block styles, and fonts are set from theme options.

DENSE TEACHING

Two-column frames pack objectives, examples, and pitfalls.

Odoo 17

ORM

Fields

CLI

1 001 – 005

Odoo Core Development · teaching block 1

001 Install Odoo 17 on Ubuntu 22.04

Objectives, key points, and lab target

Objectives

- Prepare Ubuntu 22.04 for Odoo 17 development.
- Install system packages, Python dependencies, Node tooling, and wkhtmltopdf.
- Connect the installation to a PostgreSQL role.

Key points

- Document every system package required by the Odoo instance.
- Use a Python virtual environment for source-based installs.
- Prepare PostgreSQL before first Odoo startup.
- Test PDF rendering as part of installation validation.

Lab target

Install Odoo 17 from source on Ubuntu 22.04, activate the virtual environment, and run `odoo-bin --version`.

001 Install Odoo 17 on Ubuntu 22.04

Teaching notes

An Odoo installation is a Python application, a PostgreSQL database client, a set of system libraries, and a front-end asset toolchain. Treat the server setup as part of the application, because missing libraries often appear later as PDF, image, or asset errors.

Ubuntu 22.04 is a common base for Odoo 17 because it provides a stable Python and package ecosystem. Even in training, use a documented setup so students can reproduce bugs and compare results.

A virtual environment keeps Odoo's Python packages separate from the operating system. This matters when multiple Odoo versions or experiments live on the same machine.

Also remember

- PostgreSQL should already have a dedicated role for Odoo. The role usually needs CREATEDB in development, while...
- wkhtmltopdf remains important for many Odoo PDF reports. Use a known compatible package and test report rendering...

001 Install Odoo 17 on Ubuntu 22.04

Example and pitfalls

Source install outline

This prepares a source checkout suitable for development and keeps Python dependencies inside the local virtual environment.

```
sudo apt update
sudo apt install -y git python3.10-venv python3-dev build-essential

git clone --depth 1 --branch 17.0 https://github.com/odoo/odoo
cd /opt/odoo/odoo17
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip wheel
pip install -r requirements.txt
```

Common mistakes

- Installing Python packages globally and later mixing Odoo versions.
- Skipping wkhtmltopdf validation until the first customer invoice must be printed.
- Using the postgres superuser as the Odoo database user.
- Starting from too many addons before the base installation is proven.

001 Install Odoo 17 on Ubuntu 22.04

Extra patterns

Install checklist

- System packages installed
- Python virtual environment active
- requirements.txt installed
- PostgreSQL role can connect

002 Start odoo-bin for the First Time

Objectives, key points, and lab target

Objectives

- Run Odoo from the source tree with explicit options.
- Understand addons paths, database host options, and startup logs.
- Use `--stop-after-init` for controlled initialization.

Key points

- `odoo-bin` is the source checkout entry point.
- Addons path order affects module discovery.
- `--stop-after-init` is essential for scripted installs and upgrades.
- Read startup logs from the first error forward.

Lab target

Start Odoo once with explicit CLI options and once with a config file, then compare the first 30 log lines.

002 Start odoo-bin for the First Time

Teaching notes

odoo-bin is the main entry point for source-based Odoo. Running it manually is the best way to learn what the server is loading before hiding it behind systemd or an IDE.

The addons path determines which modules Odoo can discover. A wrong path can make a valid custom module invisible, while duplicate module names across paths can create confusing behavior.

Database options tell Odoo how to reach PostgreSQL. In local development, host 127.0.0.1 with a dedicated user is explicit and avoids peer authentication surprises.

Also remember

- Startup logs show configuration, addons discovery, database connections, HTTP routes, and module loading. Senior...
- --stop-after-init is useful for installing or upgrading modules from the CLI and then exiting. It gives deterministic...

002 Start odoo-bin for the First Time

Example and pitfalls

Create and start a training database

The first command initializes the database and exits; the second starts the server using a configuration file.

```
./odoo-bin      --addons-path=addons      --db_host=127.0.0.1  
./odoo-bin -c /etc/odoo17.conf --database=odoo17_train
```

Common mistakes

- Letting Odoo guess database settings and then debugging the wrong authentication path.
- Putting a custom addons path before core addons without understanding name collisions.
- Ignoring warnings during startup because the web UI appears to load.
- Using --init repeatedly when --update is the intended operation.

003 Odoo CLI Commands Overview

Objectives, key points, and lab target

Objectives

- Recognize the most useful odoo-bin command-line options.
- Install, update, test, scaffold, and shell from the CLI.
- Choose command options appropriate for development and maintenance.

Key points

- -c selects the configuration file.
- -d selects the database.
- -i installs modules; -u upgrades modules.
- --stop-after-init makes install or upgrade commands exit.
- odoo-bin shell gives direct ORM access.

Lab target

Run --help, scaffold a throwaway module, and write the exact command that would update it in a named database.

003 Odoo CLI Commands Overview

Teaching notes

The Odoo CLI is more than a way to start the HTTP server. It is also how developers install modules, run upgrades, create scaffolds, execute tests, open shells, and control database selection.

Common options such as `-c`, `-d`, `-i`, `-u`, `--addons-path`, and `--stop-after-init` are the daily vocabulary of Odoo development. Knowing them makes deployments repeatable instead of dependent on browser clicks.

The shell command opens an Odoo environment in Python. It is powerful for diagnostics, but it can change real data, so use it with the same care as `psql`.

Also remember

- Test options allow module tests to run without manually clicking through workflows. A serious custom module should...
- CLI commands should be copied into documentation, deployment scripts, or CI jobs once they are proven. If a release...

003 Odoo CLI Commands Overview

Example and pitfalls

Daily CLI commands

These commands cover the most common lifecycle actions: inspect, install, upgrade, debug, scaffold, and test.

```
./odoo-bin --help
./odoo-bin -c odoo.conf -d odoo17_training -i sale --$
./odoo-bin -c odoo.conf -d odoo17_training -u my_modul
./odoo-bin shell -c odoo.conf -d odoo17_training
./odoo-bin scaffold academy_course /opt/odoo/custom_ad
./odoo-bin -c odoo.conf -d odoo17_training --test-enab
```

Common mistakes

- Using `-i` on a module that is already installed when `-u` is needed.
- Running shell commands against the wrong database.
- Forgetting `--stop-after-init` in scripts and leaving a server process running.
- Assuming browser-based module upgrades are equivalent to documented release commands.

004 The Odoo Configuration File

Objectives, key points, and lab target

Objectives

- Create an `odoo.conf` suitable for development.
- Configure database, addons, logging, workers, and admin password options.
- Keep secrets and environment-specific values out of source control.

Key points

- Use `odoo.conf` to make startup repeatable.
- `admin_passwd` controls database manager actions.
- `addons_path` must include every module location in the right order.
- Keep production secrets outside versioned code.

Lab target

Create a local `odoo.conf`, start Odoo with `-c`, and prove from the logs that your custom addons path was loaded.

004 The Odoo Configuration File

Teaching notes

The configuration file turns a long command line into a repeatable server definition. It should make the runtime obvious to another developer who has never used your laptop.

Database settings define where Odoo connects and which credentials it uses. In development they are often explicit, while production may use environment management or secret files around the config.

`addons_path` is one of the most important settings in the file. It must include core addons, enterprise addons if used, and custom addons in a deliberate order.

Also remember

- `admin_passwd` protects database management operations such as creating, duplicating, dropping, and backing up databases...
- Worker and limit settings should reflect the deployment mode. Development commonly uses one process with live reload,...

004 The Odoo Configuration File

Example and pitfalls

Development odoo.conf

This sample is explicit enough for training while still showing which values should be changed for real environments.

```
[options]
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/custom
data_dir = /var/lib/odoo17
db_host = 127.0.0.1
db_port = 5432
db_user = odoo17
db_password = odoo17
admin_passwd = change-this-dev-password
log_level = info
logfile = /var/log/odoo/odoo17.log
# ...
```

Common mistakes

- Committing real db_password or admin_passwd values.
- Forgetting one addons path and wondering why a module does not appear.
- Using production worker settings for local debugging.
- Changing config values but starting Odoo with a different -c file.

005 Database Manager URLs and Disabling Database Manager

Objectives, key points, and lab target

Objectives

- Use the Odoo database manager routes safely in development.
- Understand the risk of exposing database management in production.
- Disable database manager access with configuration.

Key points

- `/web/database/manager` exposes administrative database actions.
- `admin_passwd` is the master password for these operations.
- `list_db = False` hides database listing and is common in production.
- Reverse proxies can block database manager routes.

Lab target

Open the database manager in a lab instance, create a database, then set `list_db = False` and confirm the list no longer appears.

005 Database Manager URLs and Disabling Database Manager

Teaching notes

Odoo includes browser routes for creating, duplicating, restoring, backing up, and dropping databases. They are convenient in training but must be treated as administrative tools.

The database manager is usually reached through `/web/database/manager`, with related routes for create, backup, restore, duplicate, and drop. Access is protected by the master password, but exposure still increases risk.

In production, many teams disable `list_db` and block database manager routes at the reverse proxy. The principle is simple: customers should reach the application, not database administration screens.

Also remember

- `dbfilter` also matters because it controls which database a request can select. A good `dbfilter` prevents one Odoo...
- The master password must be strong and private if database manager remains enabled. It authorizes destructive actions,...

005 Database Manager URLs and Disabling Database Manager

Example and pitfalls

Production-oriented database manager settings

The config hides database listing and the reverse proxy blocks database manager routes before they reach Odoo.

```
[options]
admin_passwd = use-a-secret-manager-value
list_db = False
dbfilter = ^%d$
proxy_mode = True

# Nginx example
location ~* /web/database/ {
    deny all;
}
```

Common mistakes

- Leaving database manager visible on a public production domain.
- Using a weak admin_passwd because normal users cannot see it after login.
- Setting dbfilter incorrectly and making valid databases unreachable.
- Assuming list_db = False replaces network and proxy hardening.

2 006 – 010

Odoo Core Development · teaching block 2

006 Database CLI and Upgrade CLI Workflows

Objectives, key points, and lab target

Objectives

- Create, initialize, update, and test databases from the command line.
- Use upgrade commands safely for module development.
- Separate database operations from browser habits.

Key points

- `-i` is for installation; `-u` is for updates.
- Always specify the target database.
- `--stop-after-init` makes commands automation-friendly.
- Upgrade workflows should include backup and smoke testing.

Lab target

Install a scaffolded module with `-i`, change its manifest or view, then update it with `-u` and `--stop-after-init`.

006 Database CLI and Upgrade CLI Workflows

Teaching notes

Database work in Odoo should be scriptable. A command that creates a database, installs modules, and exits is easier to review than a sequence of browser clicks.

The `-i` option installs modules into a database and should be used for first installation. The `-u` option updates already installed modules and is the normal path after changing Python fields, views, security, or data files.

`--stop-after-init` turns Odoo into a batch command. It loads the registry, performs install or update work, runs enabled tests if requested, and exits with a status code.

Also remember

- For module upgrades, keep the database selected explicitly with `-d`. Accidentally upgrading a customer copy, staging...
- When tests are part of the command, failures should stop the release. Do not hide test failures behind a server that...

006 Database CLI and Upgrade CLI Workflows

Example and pitfalls

Repeatable module upgrade

The first command updates a module, and the second adds test execution for a release-quality upgrade check.

```
DB=odoo17_training  
CONF=/etc/odoo17.conf
```

```
./odoo-bin -c "$CONF" -d "$DB" -u academy_course --stop-after-init  
./odoo-bin -c "$CONF" -d "$DB" -u academy_course --test-enable
```

Common mistakes

- Running `-u` all casually on a large or customized production database.
- Not backing up before structural module upgrades.
- Forgetting that Odoo shell and upgrade commands can write real data.
- Using browser upgrades as the only deployment procedure.

```
--test-enable --stop-after-init --log-level=test
```


007 PyCharm Configuration for Odoo

Objectives, key points, and lab target

Objectives

- Configure a Python interpreter and run configuration for Odoo.
- Pass Odoo CLI parameters from the IDE.
- Debug server startup and custom module code.

Key points

- Use the same virtual environment in terminal and PyCharm.
- Set odoo-bin as the script path.
- Pass -c and -d explicitly in run parameters.
- Disable workers for straightforward breakpoint debugging.

Lab target

Create a PyCharm run configuration that starts your training database and stops at a breakpoint in a custom model method.

007 PyCharm Configuration for Odoo

Teaching notes

An IDE configuration should mirror the command line that already works. If PyCharm starts Odoo differently than the terminal, debugging becomes a second environment problem.

Point PyCharm at the virtual environment used for the Odoo checkout. This gives code completion for installed dependencies and prevents accidental imports from the system Python.

The script path is usually `odoo-bin`, and parameters should include `-c` and the target database or development flags. Working directory should be the Odoo source root unless your project layout intentionally differs.

Also remember

- Breakpoints work best when workers are disabled and code reload behavior is understood. In development, start with a...
- Mark custom addons and the Odoo source as project roots when useful for navigation. Good indexing makes model...

007 PyCharm Configuration for Odoo

Example and pitfalls

PyCharm run configuration values

These settings make the IDE run the same Odoo server that the terminal command would run.

Script path:

`/opt/odoo/odoo17/odoo-bin`

Parameters:

`-c /etc/odoo17.conf -d odoo17_training --dev=xml,qweb`

Working directory:

`/opt/odoo/odoo17`

Environment:

`# ...`

Common mistakes

- Using a different interpreter in PyCharm than the one used to install requirements.txt.
- Debugging with workers enabled and wondering why breakpoints are skipped.
- Leaving the database name empty and attaching to the wrong database.
- Changing PyCharm parameters without updating the documented CLI command.

008 Scaffold a Module and Use Multiple Addons Paths

Objectives, key points, and lab target

Objectives

- Generate a module scaffold with odoo-bin.
- Understand custom module directory structure.
- Configure multiple addons paths safely.

Key points

- A module is a directory with a manifest.
- The directory name is the technical module name.
- addons_path can contain several comma-separated roots.
- Duplicate module names across paths are dangerous.

Lab target

Scaffold academy_course, add its parent directory to addons_path, update the app list, and install the module from the CLI.

008 Scaffold a Module and Use Multiple Addons Paths

Teaching notes

Scaffolding creates a conventional starting point for a custom module. It is not magic, but it saves time and teaches the files Odoo expects.

A custom module is discovered because its directory sits under one of the configured addons paths and contains a manifest file. The directory name is the technical module name used in CLI commands.

Multiple addons paths are normal in real projects. A typical setup includes community addons, enterprise addons, first-party custom addons, and sometimes vendor addons.

Also remember

- Path order matters when modules share a technical name. Avoid duplicate names entirely because they make upgrades and...
- The scaffold contains models, views, security, controllers, demo, and tests placeholders depending on the template....

008 Scaffold a Module and Use Multiple Addons Paths

Example and pitfalls

Scaffold and configure paths

The scaffold command creates the module directory, and the config makes Odoo discover it.

```
mkdir -p /opt/odoo/custom_addons
./odoo-bin scaffold academy_course /opt/odoo/custom_addons

# odoo.conf
addons_path = /opt/odoo/odoo17/addons,/opt/odoo/enterprise17/addons

./odoo-bin -c /etc/odoo17.conf -d odoo17_training
```

Common mistakes

- Putting the module directory itself in addons_path instead of its parent directory.
- Using spaces around paths that are later copied incorrectly into service files.
- Keeping duplicate technical module names in different paths.
- Installing a scaffold before reviewing its manifest, security, and views.

009 Manifest Files, Dependencies, and Auto Install Modules

Objectives, key points, and lab target

Objectives

- Write a correct `__manifest__.py` for Odoo 17.
- Declare dependencies and data files deliberately.
- Use `auto_install` only for glue modules that should install automatically.

Key points

- `depends` declares modules required before this module loads.
- data file order is load order.
- `auto_install` is for glue modules.
- Keep manifests accurate and minimal.

Lab target

Create a manifest for a training module that depends on mail, includes security before views, and installs cleanly from the CLI.

009 Manifest Files, Dependencies, and Auto Install Modules

Teaching notes

The manifest is the module contract Odoo reads before loading code and data. It tells Odoo the module's name, version, dependencies, category, data files, assets, license, and install behavior.

Dependencies control load order and registry availability. If your model inherits `sale.order`, the `sale` module belongs in `depends`; relying on a user's installed apps is fragile.

Data file order matters because XML records can reference records loaded earlier in the same module or in dependencies. Security files typically load before views that expose the model.

Also remember

- `auto_install` is for modules that connect other modules when their dependencies are present. It is not a shortcut to...
- The manifest should be boring and accurate. Inflated dependencies slow installations, while missing dependencies...

009 Manifest Files, Dependencies, and Auto Install Modules

Example and pitfalls

Odoo 17 manifest

The manifest declares precise dependencies, ordered data files, and normal install behavior for an application module.

```
{
    "name": "Academy Course Management",
    "version": "17.0.1.0.0",
    "category": "Education",
    "summary": "Manage training courses and sessions",
    "depends": ["base", "mail", "product"],
    "data": [
        "security/ir.model.access.csv",
        "views/academy_course_views.xml",
        "views/academy_session_views.xml",
    ]
    # ...
}
```

Common mistakes

- Forgetting a dependency because the module happens to be installed in the developer...
- Loading views before security access rules.
- Setting `auto_install` to `True` on a normal business application module.
- Leaving `installable` `False` after copying a scaffold.

010 Custom Module Structure

Objectives, key points, and lab target

Objectives

- Organize Python, XML, security, data, demo, and test files.
- Import models correctly through `__init__.py` files.
- Keep module ownership clear as the codebase grows.

Key points

- `__init__.py` imports are required for model registration.
- `security/ir.model.access.csv` is usually loaded before views.
- data is functional; demo is sample content.
- Tests should travel with the module.

Lab target

Create the directories for `academy_course`, add import files, and confirm a simple model appears in Settings > Technical > Models.

O10 Custom Module Structure

Teaching notes

A module structure is a map for both Odoo and future developers. Consistent directories reduce cognitive load when a bug report mentions a model, view, security rule, or scheduled action.

Python packages must be imported for model classes to register. The top-level `__init__.py` imports models, and `models/__init__.py` imports the individual Python files.

Security files define who can access the model and under what record rules. Without access rights, a beautiful view can still fail with an access error as soon as a normal user opens it.

Also remember

- Views describe the user interface and should reference model fields that already exist at module load time. XML ids...
- Data and demo files serve different purposes. Data is normally required for the module to work, while demo should be...

010 Custom Module Structure

Example and pitfalls

Module layout

The layout separates responsibilities and shows the import chain Odoo needs to register models.

```
academy_course/  
  __init__.py  
  __manifest__.py  
  models/  
    __init__.py  
    academy_course.py  
    academy_session.py  
  security/  
    ir.model.access.csv  
    academy_security.xml  
# ...
```

Common mistakes

- Creating a Python model file but forgetting to import it.
- Putting required records in demo data and then missing them in production.
- Letting one large models.py file grow without domain boundaries.
- Adding views without corresponding access rights for target users.

3 011 – 015

Odoo Core Development · teaching block 3

011 `_name`, `_table`, and Model Identity

Objectives, key points, and lab target

Objectives

- Define a new Odoo model with `_name`.
- Understand default table naming and when to use `_table`.
- Choose technical names that support long-term maintenance.

Key points

- `_name` identifies the model in the registry.
- Default table name replaces dots with underscores.
- `_table` overrides the physical table name.
- Changing model identity after release requires migration.

Lab target

Create `academy.course`, install the module, and inspect the generated `academy_course` table in PostgreSQL.

011 `_name`, `_table`, and Model Identity

Teaching notes

`_name` is the model's technical identifier in the Odoo registry. It is used in Python, XML actions, security files, domains, external APIs, and logs.

By default, Odoo turns dots in `_name` into underscores for the PostgreSQL table name. A model named `academy.course` normally stores rows in `academy_course`.

`_table` can override the physical table name. Use it rarely, usually for legacy integration, SQL views, or situations where a stable existing table name must be preserved.

Also remember

- Technical names should be unique and module-scoped. A good name such as `academy.course` is clearer than `course.course...`
- Changing `_name` after data exists is a migration, not a rename in one file. Security XML ids, views, relations,...

011 `_name`, `_table`, and Model Identity

Example and pitfalls

Model identity

The class defines a new registry model and uses an explicit table name that matches Odoo's default convention.

```
from odoo import fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"
    _table = "academy_course"

    name = fields.Char(required=True)
    code = fields.Char(required=True, index=True)
    # ...
```

Common mistakes

- Renaming `_name` in a released module without a migration plan.
- Using generic model names that collide with vendor or future modules.
- Setting `_table` unnecessarily and hiding the default naming convention.
- Forgetting `_description` and making technical screens harder to read.

012 Field Overview and Simple Field Options

Objectives, key points, and lab target

Objectives

- Recognize common field declaration options.
- Use required, readonly, index, help, default, copy, and tracking deliberately.
- Understand how fields connect Python models to database columns and views.

Key points

- Field declarations carry storage, UI, and ORM metadata.
- Indexes should match real searches.
- readonly is not a security mechanism.
- copy=False is important for unique references and workflow state.

Lab target

Add five fields with different options to `academy.course` and confirm their metadata with `fields_get` in Odoo shell.

012 Field Overview and Simple Field Options

Teaching notes

Odoo fields describe both storage and business semantics. A field declaration can create a database column, expose metadata to views, enable search behavior, and participate in tracking or computed logic.

`required` enforces input at the ORM and user interface level, while SQL constraints may still be needed for deep invariants. `index` improves common search patterns but adds database write cost.

`readonly` affects how the field is edited through views and ORM conventions, but it is not a security boundary by itself. Access rights and record rules remain the real permission controls.

Also remember

- default values can be constants, callables, or context-driven helpers. Use defaults to reduce user work, but avoid...
- copy controls whether a field is duplicated when a record is copied. This is important for sequence numbers, external...

012 Field Overview and Simple Field Options

Example and pitfalls

Common field options

The fields demonstrate common options that affect validation, indexing, duplication, help text, defaults, and chatter tracking.

```
from odoo import fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"
    _inherit = ["mail.thread"]
    _description = "Academy Course"

    name = fields.Char(required=True, tracking=True)
    code = fields.Char(required=True, index=True, copy=False)
    # ...
```

Common mistakes

- Adding `index=True` to fields that are rarely searched.
- Expecting `readonly=True` to stop writes from server-side code.
- Letting `copy=True` duplicate external identifiers.
- Tracking every field and making chatter unusable.

013 Text-Oriented Fields: Char, Text, and Html

Objectives, key points, and lab target

Objectives

- Choose between Char, Text, and Html fields.
- Use size, translate, sanitize, and help options appropriately.
- Render text fields safely in views.

Key points

- Char is for short structured text.
- Text is for long plain text.
- Html is for rich formatted content and needs sanitization awareness.
- `translate=True` belongs on user-facing language content.

Lab target

Add Char, Text, and Html fields to a model, place them in a form view, and compare how each appears in the UI.

013 Text-Oriented Fields: Char, Text, and Html

Teaching notes

Char is for short values such as names, codes, references, phone numbers, and labels. It communicates that the content is concise and often searchable.

Text is for longer plain text such as internal notes, descriptions, and instructions. It does not imply rich formatting, which keeps content simpler for exports and integrations.

Html stores rich text and is common for website content, email bodies, terms, and formatted descriptions. Because it can contain markup, sanitization and trust boundaries matter.

Also remember

- `translate=True` is useful for user-facing labels or descriptions that should differ by language. Do not mark...
- The widget chosen in XML affects the editing experience. A Text field can use a textarea-like widget, while Html...

013 Text-Oriented Fields: Char, Text, and Html

Example and pitfalls

Course text fields and XML

The model separates searchable codes, plain descriptions, and rich syllabus content instead of using one text type for everything.

```
class AcademyCourse(models.Model):
    _name = "academy.course"
    _description = "Academy Course"

    name = fields.Char(required=True, translate=True)
    code = fields.Char(required=True, size=32, index=True)
    description = fields.Text(translate=True)
    syllabus_html = fields.Html(string="Syllabus", sanitize=True, translate=True)

<!-- views/academy_course_views.xml -->
# ...
```

Common mistakes

- Using Html for plain operational notes.
- Making external codes translatable.
- Expecting a Char size to replace a real business validation.
- Rendering user-provided HTML without considering sanitization.

014 Boolean, Selection, Integer, Float, and Json Fields

Objectives, key points, and lab target

Objectives

- Model flags, states, counts, quantities, and flexible payloads.
- Choose numeric fields carefully for business meaning.
- Use Json fields without replacing relational design.

Key points

- Use Selection for controlled workflow states.
- Use Integer for whole counts and Float for non-money quantities.
- Use Monetary for currency amounts instead of bare Float.
- Json is flexible but weaker than relational fields for core data.

Lab target

Create a state field with three values, add a button method that changes the state, and store one sample integration payload in a json field.

014 Boolean, Selection, Integer, Float, and Json Fields

Teaching notes

Boolean fields represent yes-or-no state and are often used for toggles such as active, published, billable, or requires approval. They are simple, but too many booleans can hide a workflow that should be a Selection.

Selection fields represent a controlled set of values. They are ideal for workflow state when the allowed values are known and transitions can be managed in methods.

Integer fields are appropriate for counts and whole-number settings. Float fields are appropriate for approximate decimal quantities, but financial amounts usually need Monetary fields.

Also remember

- Float precision in Odoo is normally controlled through decimal precision settings or digits. Do not assume Python...
- Json fields store structured Python dictionaries and lists as JSON data. They are useful for integration payloads,...

O14 Boolean, Selection, Integer, Float, and Json Fields

Example and pitfalls

Operational scalar fields

The fields show a workflow state, whole-number capacity, decimal duration, and optional integration metadata.

```
class AcademySession(models.Model):
    _name = "academy.session"
    _description = "Academy Session"

    active = fields.Boolean(default=True)
    state = fields.Selection(
        [("draft", "Draft"), ("confirmed", "Confirmed"), ("done", "Done")],
        default="draft",
        required=True,
    )
    # ...
```

Common mistakes

- Using several booleans where one Selection state would be clearer.
- Using Float for currency amounts that must reconcile.
- Storing searchable operational data only inside Json.
- Changing Selection keys without migrating existing database values.

015 Date and Datetime Fields

Objectives, key points, and lab target

Objectives

- Use Date and Datetime fields correctly in Odoo 17.
- Understand UTC storage and user timezone conversion.
- Write safe defaults and domains for date ranges.

Key points

- Date has no time; Datetime has timestamp semantics.
- Odoo stores datetimes in UTC.
- Use callable defaults such as `fields.Date.today`.
- Use half-open ranges for Datetime domains.

Lab target

Add Date and Datetime fields, create records from two user timezones, and compare the UI value to the raw PostgreSQL value.

015 Date and Datetime Fields

Teaching notes

Date fields store calendar dates without time of day. They are appropriate for birthdays, accounting dates, deadlines, validity dates, and business days.

Datetime fields store a timestamp and Odoo treats them as UTC internally. The web client displays them in the user's timezone, which is helpful but can surprise developers inspecting raw PostgreSQL values.

Use `fields.Date.today` and `fields.Datetime.now` for defaults rather than evaluating Python date functions at import time. Defaults must run when the record is created, not when the server starts.

Also remember

- Date range domains should be explicit about inclusive and exclusive boundaries. For Datetime fields, a half-open range...
- Context-aware date helpers are useful when user timezone matters for business interpretation. A scheduled action...

015 Date and Datetime Fields

Example and pitfalls

Date fields and safe domains

The example uses callable defaults and a half-open monthly range that avoids boundary confusion.

```
class AcademySession(models.Model):
    _name = "academy.session"

    start_date = fields.Date(default=fields.Date.today)
    start_at = fields.Datetime(default=fields.Datetime.now)
    deadline = fields.Date()

    def sessions_this_month(self):
        start = fields.Date.start_of(fields.Date.today(), "month")
        end = fields.Date.add(start, months=1)
        # ...
```

Common mistakes

- Writing `default=datetime.now()` with parentheses in the field declaration.
- Comparing raw UTC database values to a user's local expectation without conversion.
- Using Char fields for dates.
- Using `<= end_of_day` strings for Datetime ranges instead of a clear next boundary.

4 016 – 020

Odoo Core Development · teaching block 4

016 Relational Fields: Many2one, One2many, and Many2many

Objectives, key points, and lab target

Objectives

- Model core relationships with Odoo relational fields.
- Understand inverse fields and relation tables.
- Use ondelete, domain, context, and check_company carefully.

Key points

- Many2one stores the foreign key.
- One2many depends on an inverse Many2one.
- Many2many uses a relation table.
- Choose ondelete behavior based on business data ownership.

Lab target

Create course, session, and tag models, then inspect the generated foreign key column and Many2many relation table.

016 Relational Fields: Many2one, One2many, and Many2many

Teaching notes

Relational fields are the heart of Odoo's data model. Most business workflows are built by connecting documents, partners, companies, products, users, and accounting records through relationships.

Many2one stores a foreign key to one related record. It is the most common relation and often appears in database tables as a column ending with `_id`.

One2many is the inverse view of a Many2one. It does not store a separate list in the parent table; it shows child records whose inverse field points back to the parent.

Also remember

- Many2many stores pairs of ids in a relation table. It is useful for tags, attendees, allowed users, and other cases...
- ondelete controls what should happen when a referenced record is removed. In business systems, restrict or set null is...

016 Relational Fields: Many2one, One2many, and Many2many

Example and pitfalls

Course relationships

The parent course sees sessions through One2many, while each session stores the real course_id foreign key.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    name = fields.Char(required=True)
    session_ids = fields.One2many("academy.session", "course_id", string="Sessions")
    tag_ids = fields.Many2many("academy.tag", string="Tags")

class AcademySession(models.Model):
    _name = "academy.session"
    # ...
```

Common mistakes

- Defining One2many without a valid inverse Many2one field.
- Using cascade deletion on records that have audit or accounting meaning.
- Expecting a Many2many to preserve ordering like line items.
- Using UI domains as the only protection for business integrity.

017 Reference and Related Fields

Objectives, key points, and lab target

Objectives

- Use Reference fields for controlled polymorphic links.
- Use Related fields to expose values from linked records.
- Decide when stored related fields are worth the cost.

Key points

- Reference fields are polymorphic but less constrained.
- Related fields expose values through relation paths.
- `store=True` improves searchability at recomputation cost.
- Prefer explicit `Many2one` when only one target model is valid.

Lab target

Add a related course field on session activity, make one version stored and one non-stored, then compare `fields_get` metadata.

017 Reference and Related Fields

Teaching notes

A Reference field can point to records from different models. It is useful for generic links, but it is weaker than a normal Many2one because the database cannot enforce a single foreign key.

Use Reference only when polymorphism is truly part of the business requirement. Many designs become simpler and safer with explicit Many2one fields to the expected model.

Related fields expose a value through a relationship path, such as `session.course_id.company_id`. They reduce duplication and keep the UI close to the data users need.

Also remember

- A non-stored related field is computed on access and does not create a column. A stored related field is searchable...
- Related fields can be readonly by default because their source is another record. If users should edit the source...

017 Reference and Related Fields

Example and pitfalls

Reference and related fields

The stored `course_id` can support search or grouping, while the instructor remains a lightweight display convenience.

```
class AcademyActivity(models.Model):
    _name = "academy.activity"

    name = fields.Char(required=True)
    resource_ref = fields.Reference(
        selection=[("academy.course", "Course"), ("academy.session", "Session")],
        string="Related Resource",
    )
    session_id = fields.Many2one("academy.session")
    course_id = fields.Many2one(related="session_id.course_id", store=True)
# ...
```

Common mistakes

- Using Reference fields when a normal Many2one would enforce the relationship.
- Storing related fields without a search or reporting reason.
- Expecting database foreign keys from Reference values.
- Making related fields editable without explaining where the value is actually stored.

018 Compute Fields

Objectives, key points, and lab target

Objectives

- Write computed fields with correct dependencies.
- Choose between stored and non-stored compute fields.
- Use inverse and search methods when the business case requires them.

Key points

- Use `@api.depends` accurately.
- Assign a value for every record in the compute method.
- `store=True` trades recomputation cost for search and grouping benefits.
- Inverse and search methods are advanced tools for specific cases.

Lab target

Add `session_count` to `academy.course`, update sessions, and confirm the count changes after module upgrade.

018 Compute Fields

Teaching notes

Computed fields derive their value from other fields or records. They keep business logic in Python instead of forcing users or integrations to maintain redundant values manually.

@api.depends tells Odoo which changes should trigger recomputation. Missing dependencies create stale values, while overly broad dependencies waste time during writes.

Non-stored computed fields are calculated when read. Stored computed fields create database columns and are recomputed when dependencies change, which makes them searchable and groupable in more cases.

Also remember

- Every record in self must receive a value in the compute method. Compute methods that accidentally assign only some...
- Inverse methods allow a computed field to be edited by translating the entered value back to source fields. Use them...

018 Compute Fields

Example and pitfalls

Stored computed field

The compute method assigns every course a count and stores the result for efficient display and search.

```
from odoo import api, fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"

    session_ids = fields.One2many("academy.session", "course_id")
    session_count = fields.Integer(compute="_compute_session_count", store=True)

    @api.depends("session_ids")
    # ...
```

Common mistakes

- Forgetting @api.depends and wondering why values are stale.
- Assigning a computed value only inside an if branch.
- Setting store=True on expensive fields without considering write impact.
- Using compute fields to hide data that should be normalized into real relational fields.

019 Monetary, Binary, and Image Fields

Objectives, key points, and lab target

Objectives

- Represent currency amounts with Monetary fields.
- Store files and images with Binary and Image fields.
- Understand attachment storage and UI widgets.

Key points

- Use Monetary with a currency field for money.
- Binary fields often store data as attachments.
- Image fields support image-specific UI and processing.
- Database and filestore backups must stay together.

Lab target

Add a fee, currency, brochure, and image to academy.course, then upload a file and confirm an ir.attachment was created.

019 Monetary, Binary, and Image Fields

Teaching notes

Monetary fields pair an amount with a currency field. This is the correct pattern for prices, fees, budgets, and totals that users expect to format by currency.

A Monetary field normally references a `res.currency` record through `currency_field`. Without a clear currency, an amount is just a number and can be misread in multi-company environments.

Binary fields store file-like data and often use `attachment=True` so data is stored as an `ir.attachment` rather than directly in the model table. This keeps large payloads out of frequently scanned business tables.

Also remember

- Image fields build on binary storage with image-specific resizing and widgets. They are appropriate for logos, profile...
- Files and images affect backup strategy because Odoo attachments commonly live in the filestore. A database dump...

O19 Monetary, Binary, and Image Fields

Example and pitfalls

Money and files

The example links amounts to company currency and stores course files through Odoo's attachment mechanism.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    currency_id = fields.Many2one(
        "res.currency",
        default=lambda self: self.env.company.currency_id,
        required=True,
    )
    fee = fields.Monetary(currency_field="currency_id")
    brochure = fields.Binary(attachment=True)
    # ...
```

Common mistakes

- Using Float for prices instead of Monetary.
- Forgetting the currency field in multi-company data.
- Backing up the database but not the filestore.
- Storing large files directly on heavily used models without considering attachment...

O20 Default ORM Methods and Recordsets

Objectives, key points, and lab target

Objectives

- Understand recordsets and default model method style.
- Use browse, create, write, copy, and unlink safely.
- Recognize when methods operate on multiple records.

Key points

- self is usually a recordset.
- browse does not guarantee records still exist.
- Core lifecycle methods should be extended carefully.
- Prefer ORM operations unless direct SQL is deliberately justified.

Lab target

In Odoo shell, create three courses, batch update them with write, duplicate one, and check exists on a deleted id.

O20 Default ORM Methods and Recordsets

Teaching notes

Odoo's ORM represents records as recordsets, even when there is only one record. This design lets methods work consistently on zero, one, or many records.

Model methods receive self as a recordset or model environment depending on how they are called. A method should be clear about whether it supports multiple records or requires exactly one.

browse creates a recordset from ids without immediately verifying that every id exists. exists is often used later when the code must discard deleted or inaccessible records.

Also remember

- create, write, copy, and unlink are core lifecycle methods. They are powerful extension points but also high-risk...
- The ORM handles access rules, field conversions, computed fields, invalidation, and relational commands. Direct SQL...

020 Default ORM Methods and Recordsets

Example and pitfalls

Recordset basics in shell

The shell example shows recordset search, batch write, existence checking, copy, and create through the ORM.

```
courses = env["academy.course"].search([("active", "=", 1)])
courses.write({"state": "published"})

course = env["academy.course"].browse(42)
if course.exists():
    duplicate = course.copy({"name": "%s (Copy)" % course.name})

env["academy.course"].create({
    "name": "Odoo Developer Bootcamp",
    "code": "ODOO-BOOT",
    # ...
})
```

Common mistakes

- Assuming self always contains exactly one record.
- Looping over ids and performing one search per id.
- Using direct SQL for changes that need access rules or recomputation.
- Calling browse and then assuming the record exists in the database.

5 021 – 025

Odoo Core Development · teaching block 5

021 Override create and write

Objectives, key points, and lab target

Objectives

- Override create with `@api.model_create_multi`.
- Override write without breaking batch behavior.
- Call super and preserve the inheritance chain.

Key points

- Use `@api.model_create_multi` for create overrides.
- `write` applies vals to every record in self.
- Capture old values before super when needed.
- Call super to preserve installed module behavior.

Lab target

Override create to assign a sequence and write to stamp a publication datetime only when the state first becomes published.

021 Override create and write

Teaching notes

create and write are the most common lifecycle overrides in custom modules. They are also easy to damage because they sit on hot paths used by imports, forms, automated actions, and other modules.

In Odoo 17, create should normally use `@api.model_create_multi` so one call can create several records efficiently. Code that assumes a single vals dictionary can slow imports or break batch creation.

write receives one vals dictionary applied to every record in self. If behavior depends on old values, capture them before super because after super the recordset reflects the new data.

Also remember

- Always call super unless you are intentionally replacing core behavior, which is rare. The inheritance chain may...
- Validation in create or write should be targeted and explainable. Prefer constraints for invariant rules and use...

021 Override create and write

Example and pitfalls

Safe create and write overrides

The create override supports batch creation, and write captures old state before calling super.

```
from odoo import api, fields, models

class AcademyCourse(models.Model):
    _name = "academy.course"

    name = fields.Char(required=True)
    code = fields.Char(copy=False)
    published_at = fields.Datetime(copy=False)
    state = fields.Selection([("draft", "Draft"), ("published", "Published")], default="draft")
    # ...
```

Common mistakes

- Using `@api.model` and a single vals dict for batch create code.
- Forgetting to return the result of super.
- Reading old values after super and losing the transition information.
- Creating recursive writes without guards.

022 Custom Methods, copy, and unlink

Objectives, key points, and lab target

Objectives

- Write business methods that operate on recordsets.
- Override copy to reset duplicated values.
- Override unlink to protect important records.

Key points

- Custom methods should express business actions.
- Use `ensure_one` only for truly single-record actions.
- Override copy to reset duplicated business identifiers.
- Prefer archive over unlink for historical business records.

Lab target

Add a publish button method, prevent deletion of published records, and verify duplicate records return to draft state.

022 Custom Methods, copy, and unlink

Teaching notes

Custom methods should express business actions in the language of the module. Button handlers such as `action_confirm` or `action_archive` are clearer than placing workflow logic directly in views.

Methods called by buttons often require one record, but batch actions may pass many. Use `ensure_one` only when the method truly cannot operate on multiple records.

`copy` duplicates a record and lets you provide default values for the duplicate. Reset fields such as state, sequence code, external references, and counters that should not be cloned.

Also remember

- `unlink` deletes records and should be restricted when records have business history. In many Odoo workflows, archiving...
- Raise `UserError` or `ValidationError` with messages that help the user fix the problem. A stack trace or generic access...

022 Custom Methods, copy, and unlink

Example and pitfalls

Business methods and deletion protection

The methods implement a publish action, safe duplication defaults, and deletion protection for published courses.

```
from odoo import _, models
from odoo.exceptions import UserError

class AcademyCourse(models.Model):
    _name = "academy.course"

    def action_publish(self):
        for course in self:
            if not course.session_ids:
# ...
```

Common mistakes

- Calling `ensure_one` at the top of every method by habit.
- Letting copy duplicate unique business references.
- Deleting records that should remain for audit history.
- Raising technical exceptions that do not guide the user.

023 exists, search, and Domain Operators

Objectives, key points, and lab target

Objectives

- Use exists to clean recordsets.
- Write search domains with common operators.
- Combine domain logic with &, |, and !.

Key points

- search takes a domain and returns a recordset.
- Domain operators describe SQL-like comparisons and Odoo hierarchy logic.
- Prefix | means OR, & means AND, and ! means NOT.
- exists filters out missing records from a recordset.

Lab target

Write three domains: exact state, OR condition, and inactive-inclusive search using `with_context(active_test=False)`.

023 exists, search, and Domain Operators

Teaching notes

search is the main ORM method for finding records. It accepts a domain, optional offset, limit, order, and count behavior through related methods.

A domain is a list representation of logical conditions. Each tuple contains a field, an operator, and a value, while prefix operators such as | and ! combine expressions.

Common operators include =, !=, in, not in, like, ilike, child_of, >=, <=, and =?. The right operator makes code more readable and can change whether indexes are useful.

Also remember

- exists returns the subset of a recordset that still exists and is accessible. It is useful after browse on external...
- Domains are translated to SQL with ORM context, access rights, active_test, and company behavior in mind. A domain is...

023 exists, search, and Domain Operators

Example and pitfalls

Search domains

The examples show normal AND behavior, explicit OR behavior, ordering, limiting, and existence cleanup.

```
Course = self.env["academy.course"]

published = Course.search([
    ("active", "=", True),
    ("state", "=", "published"),
    ("name", "ilike", "odoo"),
], order="name", limit=20)

draft_or_expensive = Course.search([
    "|",
    # ...
```

Common mistakes

- Building domains with string concatenation instead of domain lists.
- Forgetting that `active_test` may hide inactive records.
- Using `ilike` for every search even when exact equality is intended.
- Browsing arbitrary ids and assuming all records exist.

024 search_count, read, read_group, and search_read

Objectives, key points, and lab target

Objectives

- Choose the right read method for lists, counts, and aggregates.
- Use read_group for grouped reporting.
- Avoid loading unnecessary fields or records.

Key points

- Use search_count for counts.
- Use read for dictionaries from known records.
- Use search_read for search plus field extraction.
- Use read_group for grouped aggregates.

Lab target

Build a small dashboard method that returns draft count, the ten latest published courses, and fees grouped by state.

024 search_count, read, read_group, and search_read

Teaching notes

Not every data question needs full recordsets with every field prefetched. Odoo provides specialized methods for counts, raw reads, combined search-and-read, and grouped aggregation.

`search_count` returns the number of records matching a domain without fetching all records. It is ideal for badges, warnings, and decisions that only need a count.

`read` returns dictionaries for fields on an existing recordset. It is useful for RPC-like output or integration code that needs simple structures rather than record methods.

Also remember

- `search_read` combines search and read in one call and is common in controllers and API-style endpoints. It should still...
- `read_group` performs grouped aggregation similar to SQL GROUP BY while respecting ORM access and context. It is useful...

024 search_count, read, read_group, and search_read

Example and pitfalls

Reading and grouping

The methods answer count, list, and grouped reporting questions without loading more data than needed.

```
Course = self.env["academy.course"]

draft_count = Course.search_count([("state", "=", "draft")])

rows = Course.search_read(
    [("state", "=", "published")],
    fields=["name", "code", "fee"],
    limit=10,
    order="name",
)
# ...
```

Common mistakes

- Using `len(search(domain))` instead of `search_count` for large datasets.
- Calling `search_read` without a limit in controller code.
- Expecting `read_group` output to look exactly like SQL rows.
- Fetching all fields when only names and ids are required.

025 name_create, default_get, _name_search, and get_view

Objectives, key points, and lab target

Objectives

- Customize quick creation and default values.
- Improve Many2one search behavior with _name_search.
- Understand when get_view customization is appropriate.

Key points

- name_create handles quick creation from relational fields.
- default_get returns context-aware defaults.
- _name_search improves lookup behavior.
- get_view is powerful and should be rare.

Lab target

Make academy.course searchable by code in a Many2one field and add a default company through default_get.

025 name_create, default_get, _name_search, and get_view

Teaching notes

`name_create` supports quick creation from relational widgets. It should create a valid minimal record, not bypass required business setup that the user must provide.

`default_get` computes default values for new records based on fields, context, company, user, and business rules. It is a good place for defaults that need more logic than a field declaration.

`_name_search` controls how records are found when users type in Many2one fields. It should return ids matching the user's text across meaningful fields such as code and name.

Also remember

- Search customization must respect access rules and performance. A convenient `_name_search` that scans large text fields...
- `get_view` can modify view architecture before it reaches the client. It is powerful but should be used sparingly...

025 name_create, default_get, _name_search, and get_view

Example and pitfalls

Lookup and defaults hooks

The model supports quick creation, company-aware defaults, and lookup by either course name or code.

```
from odoo import api, models

class AcademyCourse(models.Model):
    _name = "academy.course"
    _rec_name = "name"

    @api.model
    def name_create(self, name):
        record = self.create({"name": name, "code": name.upper().replace(" ", "-")[:32]})
# ...
```

Common mistakes

- Using name_create to create incomplete records that later fail business rules.
- Overriding default_get and ignoring the requested fields_list.
- Writing _name_search domains that ignore args passed by the caller.
- Using get_view for changes that normal XML inheritance can handle.

6 026 – 030

Odoo Core Development · teaching block 6

026 ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata

Objectives, key points, and lab target

Objectives

- Use recordset helper methods idiomatically.
- Inspect model metadata from the ORM.
- Avoid unnecessary loops and single-record assumptions.

Key points

- ensure_one is an assertion, not a default habit.
- filtered, mapped, sorted, and grouped express recordset transformations.
- fields_get exposes field metadata.
- get_metadata helps trace XML ids and record provenance.

Lab target

Use Odoo shell to filter published courses, map their names, group by state, and print fields_get metadata for three fields.

026 ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata

Teaching notes

Recordset helpers make Odoo code concise and expressive. They operate on recordsets and usually preserve ORM behavior better than manually building lists of ids.

`ensure_one` asserts that a method is working with exactly one record. It is appropriate for single-record actions and computed display helpers, but it should not be used to avoid thinking about batch behavior.

`filtered` keeps records matching a predicate or field expression, `mapped` extracts fields or calls, `sorted` orders records, and `grouped` partitions records by a key. These methods make business code read closer to the business question.

Also remember

- `fields_get` returns field metadata such as type, string, required, readonly, selection, and help. It is useful for...
- `get_metadata` returns technical metadata such as XML ids and creation or update information. It helps when diagnosing...

026 ensure_one, filtered, mapped, sorted, grouped, fields_get, and get_metadata

Example and pitfalls

Recordset helpers

The code shows common helpers and a correct single-record action that uses ensure_one intentionally.

```
courses = self.env["academy.course"].search([])

published = courses.filtered(lambda course: course.state == "published")
names = published.mapped("name")
ordered = published.sorted(key=lambda course: course.name)
by_state = courses.grouped("state")

field_info = self.env["academy.course"].fields_get(["name", "state", "tree"])
metadata = published[:1].get_metadata()

# ...
```

Common mistakes

- Calling ensure_one in methods that could naturally support batch actions.
- Using filtered on thousands of records when a search domain would be better.
- Assuming mapped always returns a recordset; scalar fields return Python lists.
- Ignoring metadata tools and hardcoding guesses about fields or XML ids.

027 search_fetch and Efficient Data Loading

Objectives, key points, and lab target

Objectives

- Use search_fetch when records and selected fields are needed together.
- Understand prefetching and field loading behavior.
- Reduce query chatter in read-heavy custom code.

Key points

- search_fetch combines search with deliberate field fetching.
- Use it when you need a recordset and known fields immediately.
- search_read is better when dictionaries are the desired output.
- Measure performance changes before making broad refactors.

Lab target

Write one method with search and one with search_fetch for the same field access pattern, then compare SQL logs in a small benchmark.

027 search_fetch and Efficient Data Loading

Teaching notes

Odoo's ORM uses caching and prefetching to reduce repetitive database access. Most normal code benefits automatically, but high-volume loops can still create avoidable query chatter.

`search_fetch` searches for records and fetches specified fields in one deliberate operation. It is useful when the next code path will read a known set of fields from every returned record.

The method keeps recordset semantics while making field-loading intent explicit. This can be cleaner than `search` followed by manual reads when the code still wants record methods.

Also remember

- Do not use `search_fetch` everywhere by habit. If code only needs ids, `search` may be enough; if it needs dictionaries...
- Performance work should be measured with logs, tests, or query counters when possible. Odoo caches can make a change...

027 search_fetch and Efficient Data Loading

Example and pitfalls

Fetch fields for a batch job

The batch job declares the fields it will use before looping over the recordset.

```
def _send_course_digest(self):
    courses = self.env["academy.course"].search_fetch(
        [("state", "=", "published")],
        ["name", "code", "fee", "currency_id"],
        limit=500,
        order="name",
    )
    for course in courses:
        body = "%s [%s] costs %s" % (course.name, course.code, course.fee)
        self.env["mail.mail"].create({
# ...
```

Common mistakes

- Using search_fetch when only a count is needed.
- Fetching many fields just in case.
- Optimizing without measuring query count or runtime.
- Replacing clear search_read API code with recordset code that callers do not need.

028 Special Relational Commands 0 through 6

Objectives, key points, and lab target

Objectives

- Use Odoo's relational command tuples correctly.
- Apply commands to One2many and Many2many fields.
- Prefer Command helpers for readable Odoo 17 code.

Key points

- 0 creates, 1 updates, 2 deletes.
- 3 unlinks, 4 links, 5 clears, 6 replaces.
- Use `odoo.fields.Command` for readable code.
- Delete and unlink are different operations.

Lab target

Create a course with two sessions using `Command.create`, add two tags with `Command.set`, then remove one tag with `Command.unlink`.

028 Special Relational Commands 0 through 6

Teaching notes

Relational commands describe changes to One2many and Many2many fields in create and write calls. They let one ORM call create, update, link, unlink, clear, or replace related records.

The classic tuple commands are numbered 0 through 6. They are compact but easy to misread, so Odoo's Command helper is preferred in modern code for clarity.

Command 0 creates a new related record, command 1 updates an existing related record, and command 2 deletes the related record. For One2many, these are common when editing line items.

Also remember

- Command 3 unlinks a relation without deleting the target record, command 4 links an existing record, command 5 clears...
- Be careful with command 2 because it deletes the related record, not just the relationship. On business documents,...

028 Special Relational Commands 0 through 6

Example and pitfalls

Command helper examples

The helper names make the intent clearer than raw numbers while still producing the same ORM commands.

```
from odoo.fields import Command

course = self.env["academy.course"].create({
    "name": "Advanced Odoo",
    "session_ids": [
        Command.create({"name": "ORM Deep Dive"}),
        Command.create({"name": "Performance Lab"}),
    ],
    "tag_ids": [
        Command.link(existing_tag_id),
    ],
    # ...
})
```

Common mistakes

- Using command 2 when command 3 was intended.
- Replacing all Many2many links with command 6 when the code meant to add one link.
- Passing raw external command lists into write without validation.
- Forgetting that One2many requires values compatible with the child model.

029 Active Archive Behavior

Objectives, key points, and lab target

Objectives

- Use the active field for archiving records.
- Understand active_test context in searches and relational fields.
- Choose archive or unlink based on business history.

Key points

- active=False archives records.
- active_test controls automatic filtering of inactive records.
- Archive preserves history better than unlink.
- active is not a workflow state field.

Lab target

Add active to academy.course, archive one record, prove normal search hides it, and prove active_test=False finds it.

029 Active Archive Behavior

Teaching notes

Many Odoo models use an active Boolean to archive records instead of deleting them. Archived records remain in the database but are hidden from normal searches and views.

The ORM context key `active_test` controls whether inactive records are filtered automatically. By default, `active_test` is usually true, so search results may exclude archived records without an explicit domain.

Archiving is safer for records referenced by historical documents. A partner, product, course, or analytic account may no longer be selectable for new work but still needs to appear on old records.

Also remember

- Views often provide Archive and Unarchive actions automatically when a model has active. This creates a consistent...
- Search domains that explicitly mention active can interact with `active_test` in confusing ways. When diagnosing...

029 Active Archive Behavior

Example and pitfalls

Archive-aware searches

The lookup includes archived records deliberately, while the archive action hides completed courses without deleting them.

```
class AcademyCourse(models.Model):
    _name = "academy.course"

    active = fields.Boolean(default=True)

    def find_any_course(self, code):
        return self.with_context(active_test=False).search([("code", "=", code)], limit=1)

    def action_archive_old_courses(self):
        old_courses = self.search([
# ...
```

Common mistakes

- Deleting historical records instead of archiving them.
- Forgetting `active_test=False` when searching for archived records.
- Using `active` to represent draft, confirmed, and done workflow states.
- Adding custom archive logic when the standard `active` behavior is enough.

030 Environment self.env and Modifying Environment

Objectives, key points, and lab target

Objectives

- Use self.env to access models, user, company, context, and cursor.
- Modify environment with sudo, with_context, with_user, with_company, and with_env.
- Understand the security impact of environment changes.

Key points

- self.env gives access to models, cursor, user, company, and context.
- with_context changes contextual behavior for a recordset.
- sudo bypasses normal access checks and must be narrow.
- with_user and with_company make user and company changes explicit.

Lab target

In Odoo shell, print env.user, env.company, and env.context, then compare search results with normal access, sudo, and active_test=False.

O30 Environment self.env and Modifying Environment

Teaching notes

self.env is the gateway to the Odoo runtime. It carries the database cursor, current user, context, registry, company information, cache, and model accessors.

self.env['model.name'] returns a model recordset bound to the same environment. This is how custom code reaches other models without importing their Python classes directly.

with_context creates a new recordset with modified context. Context can drive defaults, language, timezone, active_test, company behavior, and many model-specific flags.

Also remember

- sudo changes access rights by using superuser mode or a specific user. It is necessary for some technical operations,...
- with_user and with_company are more explicit tools when behavior should be evaluated as another user or company. They...

030 Environment self.env and Modifying Environment

Example and pitfalls

Environment patterns

The method shows context changes, narrow sudo usage, company-scoped work, and model access through self.env.

```
def action_prepare_digest(self):
    Mail = self.env["mail.mail"]
    lang = self.env.user.lang or "en_US"

    courses = self.with_context(active_test=False, lang=
        ("state", "=", "published"),
    ])

    admin_course_count = self.sudo().search_count([])

# ...
```

Common mistakes

- Using sudo everywhere instead of fixing access rights or record rules.
- Forgetting that with_context returns a new recordset.
- Using self.env.cr for direct SQL when ORM methods would be safer.
- Mixing companies in code without explicit with_company or company domains.

O30 Environment self.env and Modifying Environment

Extra patterns

Security habit

Keep sudo blocks as small as possible and never use them to hide missing business security design.

Σ You should now be able to...

Section wrap-up

- O01 Install Odoo 17 on Ubuntu 22.04
- O02 Start odoo-bin for the First Time
- O03 Odoo CLI Commands Overview
- O04 The Odoo Configuration File
- O05 Database Manager URLs and Disabling Database Manager
- O06 Database CLI and Upgrade CLI Workflows
- O07 PyCharm Configuration for Odoo
- O08 Scaffold a Module and Use Multiple Addons Paths
- Complete every lab before the next section
- Keep a command/error journal
- Prefer disposable databases for experiments
- Re-read pitfalls before production changes

Odoo 17

ORM

Fields

CLI

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 04 · Odoo Core

